# Little Man programming, super-detailed: the **Triangle**, **Memory**, and **Grade Book** machines

team wheezards — ICFPC 2026 working notes

2026-07-27

This document explains three of our contest programs: the tiny **Triangle** machine (rank 1 of 239) and the packed **Memory** machine at instruction level, then the much larger **Grade Book** machine as a system of sixteen named components. Triangle and Memory are shown both as literal `.man` grids and in our geometry-free *block-graph* notation. Grade Book instead gets a component catalogue, explicit stream protocols, unit-test boundaries, and a room-by-room optimization map. Every quoted measurement comes from the checked-in artifacts, their tests, or preserved live responses.

## Contents

# 1 The machine in ninety seconds

A `.man` program is an ASCII grid. **Rooms** are rectangles drawn with `+ - |` walls. A little man spawns at every `@` inside a room and always starts facing *East*. He carries three registers:

| | | |
|---|---|---|
| `A` | main hand | almost every operation reads or writes it |
| `B` | off hand | written *only* by `M`, `W` and `/` (proven empirically) |
| `BP` | backpack | a counter; written by `b`, `m`; branched on by `d` |

**Time** advances in ticks. On every tick, in this order: (1) every pipe shifts its values one cell forward; (2) the output room emits a finished value, the input room injects a waiting one; (3) every man *executes the operation he is standing on, then steps one cell in his heading* — unless he is halted or blocked. Two men colliding halt both. *Walking into a wall is an error that fails the whole run* — a correct program never does it (but see the Triangle section for why a program may legally be *about* to do it when the judge stops the clock).

**Pipes** are directed FIFO conveyors drawn outside rooms with `- |` bodies and arrowheads; each pipe cell holds at most one value, and values travel *one cell per tick* — a pipe of length $L$ is also a delay of $L$ ticks. `s` sends `A` into the *nearest* outgoing pipe and blocks while the pipe's entry cell is occupied; `r` receives into `A` from the nearest incoming pipe and blocks until a value has arrived at its end. "Nearest" is Manhattan distance from the instruction cell to the point where the pipe crosses the room wall. The special rooms `|I|` and `|O|` are the program's input and output: the judge feeds values through the `I` room's pipe and reads answers arriving at `O`.

**Numbers.** A bare digit sets `A` = 0..9. A backtick literal like `` `123` `` loads its number into `A` when the man walks onto the *closing* backtick (so the same cells read differently walked left-to-right and right-to-left). All arithmetic is signed 64-bit.

**Halting and scoring.** `H` halts one man forever; the run ends when every man has stopped — or earlier, the moment the judge has seen the last expected output. The score of a passing program is

$$\max(\text{width}, \text{height})^2 \times \text{average ticks},$$

lower is better; the bounding box covers the *entire* program including pipes and I/O rooms. Some problems (not these two) are footprint-only.

# 2 The geometry-free block-graph notation

The `.man` grid mixes two things: the *algorithm* (the sequence of operations the man walks over) and the *geometry* (where corridors turn). Our block-graph notation strips the geometry. A program becomes basic blocks of straight-line operations:

- `(mark L1_8_E)` names a block — auto-generated from the cell (row 1, column 8) and heading (East) where the block starts;
- operations follow in walking order; `(goto L)` is an unconditional jump (in the grid: a corridor);
- `(if NEG ZERO POS)` is the three-way branch of `X` (turn by the sign of `A`);
- `(if-bp TURN STRAIGHT)` is the two-way branch of `d` (turn clockwise if `BP` $> 0$);
- `(r p21_8)` / `(s p23_30)` name *which* pipe an `r`/`s` binds to, by the coordinates of the cell where that pipe crosses the room wall — in the grid this is implicit nearest-pipe resolution, and making it explicit is exactly what a binding audit checks;
- `(wall)` marks a path that would run into a wall — reaching it is an error, so in a correct machine it is dead code.

A note on the bare heading ops (`< ^ v >`) that still appear inside blocks: every *conditional* turn is lifted into an explicit branch form with absolute targets, and literal values are resolved, so heading carries no control-flow meaning in this notation. The remaining arrows are semantic no-ops kept deliberately, because one op is one tick: a block's length IS its cost, and every walked cell stays accounted for. Read as a language they are noise; read as a transcript with the geometry erased they are the cost model. A planned normalisation replaces each such run with `(nop` $n$`)` — an explicit "$n$ ticks pass here" — which keeps the cost model while cleaning the text, and doubles as a compile-side directive for timing-sensitive machines, where exact delays are today built by hand-counting corridor cells.

Both notations below are generated mechanically from the artifacts by our decompiler, and the decompiled form re-validates against the simulator — so what you read is what actually runs.

# 3 Triangle

**The problem.** *Output the n-th triangular number: read one integer n, output* $1 + 2 + \cdots + n$*; for* $n = 0$ *output* 0. Six public test cases, one number each. Scoring is footprint × ticks.

**Two of our machines** appear below. `triangle_02.man` ($9 \times 9$, 11 ticks, score 891) is the teaching machine: two rooms, two men, three pipes, visible blocking. The actual record holder — rank 1 of about 240 — is its smaller sibling `triangle_04.man` ($8 \times 8$, 13 ticks, score $64 \times 13 = \mathbf{832}$), explained in section 3.5: it wins with MORE ticks by exploiting the squared footprint, it deliberately sacrifices its man, and it is invisible to the plain local judge — a two-judge story worth reading.

## 3.1 The program, in both notations

```
col  012345678
  0  +-------+
  1  |@rM*+sH|
  2  +-------+
  3  +-+>^+-+v
  4  |I| >|O||
  5  +-+ ^+-+v
  6  +-------+
  7  |@1Mr}sH|
  8  +-------+
```

Two rooms (rows 0–2 and rows 6–8), each with one man; the `I` room feeds the top room through the 2-cell pipe `>^` (row 3, cols 3–4); the top room sends to the bottom room down the 3-cell pipe at column 8 (rows 3–5); the bottom room sends to `O` through the 2-cell pipe `^>` (rows 4–5, col 4). In block-graph form the geometry disappears and each man is one straight line:

```
man 1:  (mark L1_1_E) @ r M * + s H
man 2:  (mark L7_1_E) @ 1 M r } s H
```

## 3.2 Glossary — every operation this program uses

- `@` the man's start cell; ordinary space once he walks off it
- `r` receive: `A` ← next value from the nearest incoming pipe; *blocks* until one arrives
- `M` `B` ← `A` (`A` unchanged)
- `*` `A` ← `A`×`B`
- `+` `A` ← `A`+`B`
- `1` `A` ← 1
- `}` arithmetic shift right: `A` ← `A`≫`B` (sign-preserving)
- `s` send: push `A` into the nearest outgoing pipe; *blocks* while its entry cell is full
- `H` halt this man; he stays on the `H` forever

## 3.3 The algorithm, informally

The identity being computed is

$$T(n) \;=\; \frac{n(n+1)}{2} \;=\; \frac{n^2+n}{2} \;=\; (n^2+n) \gg 1,$$

and the shift is exact because $n^2 + n = n(n+1)$ is a product of consecutive integers, hence always even.

The work is split between two specialists connected by a pipe:

- **man 1** (top room) reads $n$, computes $n^2 + n$ with three register operations — `M` parks $n$ in `B`, `*` squares, `+` adds the parked $n$ — sends the result downstream, halts.
- **man 2** (bottom room) first loads the constant: `1 M` puts 1 into `B`. *He does this while man 1 is still computing* — the two run in parallel. Then he blocks on `r` until $n^2 + n$ arrives, halves it with `}` (shift right by `B` = 1), sends the answer to `O`, halts.

Why two rooms at all? A single-room version must fit `r M * + M 1 W } s H` plus turns into one box and ends up no smaller, while the two-room split lets each room be a bare one-row corridor and overlaps man 2's constant-loading with man 1's arithmetic. Note also the register discipline: man 2 sets `B`=1 *before* blocking on `r`, because `r` only writes `A` — `B` rides through the wait untouched.

### 3.4 Step by step: $n = 3$, tick by tick

The table shows the state *after* each tick: where the man stands, the operation now under his feet (executed on the *next* tick), and his registers after this tick's execution. Input: 3. Expected output: 6.

| | man 1 (top room) | | | | | man 2 (bottom room) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| tick | at | on | A | B | | at | on | A | B | |
| 1 | (1,2) | `r` | 0 | 0 | | (7,2) | `1` | 0 | 0 | |
| 2 | (1,3) | `M` | 3 | 0 | | (7,3) | `M` | 1 | 0 | |
| 3 | (1,4) | `*` | 3 | 3 | | (7,4) | `r` | 1 | 1 | |
| 4 | (1,5) | `+` | 9 | 3 | | (7,4) | `r` | 1 | 1 | |
| 5 | (1,6) | `s` | 12 | 3 | | (7,4) | `r` | 1 | 1 | |
| 6 | (1,7) | `H` | 12 | 3 | | (7,4) | `r` | 1 | 1 | |
| 7 | (1,7) | `H` | 12 | 3 | halted | (7,4) | `r` | 1 | 1 | |
| 8 | (1,7) | `H` | 12 | 3 | halted | (7,5) | `}` | 12 | 1 | |
| 9 | (1,7) | `H` | 12 | 3 | halted | (7,6) | `s` | 6 | 1 | |
| 10 | (1,7) | `H` | 12 | 3 | halted | (7,7) | `H` | 6 | 1 | |
| 11 | (1,7) | `H` | 12 | 3 | halted | (7,7) | `H` | 6 | 1 | halted |

Reading it:

- **tick 1–2:** the input value 3 travels the 2-cell input pipe while man 1 walks off `@`; his `r` fires on tick 2 (`A` = 3) with no waiting — the pipe delay was hidden behind his first step. Man 2 meanwhile loads `1` and copies it to `B` (`M` on tick 2).
- **tick 3–5, man 1:** `*` makes 9, `+` makes 12, and on tick 5 he stands on `s`.
- **tick 3–7, man 2:** he reached his `r` on tick 3 and *blocks* — position frozen at $(7, 4)$ — because nothing has arrived yet. Blocking costs him nothing: he got his constant ready early.
- **tick 6:** man 1 executes `s`: the value 12 enters the 3-cell pipe at column 8. He steps onto `H` and halts on tick 7.
- **tick 6–8:** 12 rides the pipe one cell per tick and reaches the pipe's end; man 2's `r` fires on tick 8 (`A` = 12, `B` still 1).
- **tick 9–10:** `}` halves: `A` = 6; `s` sends 6 into the output pipe.
- **tick 11:** 6 arrives at `O`. The judge has now seen every expected output and *stops the clock*: 11 ticks. (Man 2 halts on his `H` the same tick; had he been about to walk into a wall instead, it would not have mattered — ticks after the last output never run.)

Score: bounding box $9 \times 9 \Rightarrow 81$; ticks 11; $81 \times 11 = 891$. The lesson that generalises: *overlap work across men, park constants in `B` before blocking, and remember the run is only as long as the judge needs it to be.*

### 3.5 The record holder: no halt, a sacrificed man, and two judges

`triangle_04.man` beats the machine above 891 → 832 while taking *more* ticks (13 vs 11) — the score squares the bounding box, so $64 \times 13 = 832 < 81 \times 11 = 891$: losing one row and one column was worth two extra ticks. It contains **no `H` at all**.

```
col  01234567
  0  +------+
  1  |@rM*+v|
  2  |s}W1M<|
  3  +------+
  4  +-+>^  v
  5  |I| +-+<
  6  +-+ |O|
  7      +-+
```

```
(mark L1_1_E) @ r M * + v < M 1 W } s (goto __wall)
(mark __wall) (wall)
```

Trace for $n = 3$ (registers after each tick):

| tick | at | on | A | B |
|---|---|---|---|---|
| 1 | (1,2) | `r` | 0 | 0 |
| 2 | (1,3) | `M` | 3 | 0 |
| 3 | (1,4) | `*` | 3 | 3 |
| 4 | (1,5) | `+` | 9 | 3 |
| 5 | (1,6) | `v` | 12 | 3 |
| 6 | (2,6) | `<` | 12 | 3 |
| 7 | (2,5) | `M` | 12 | 3 |
| 8 | (2,4) | `1` | 12 | 12 |
| 9 | (2,3) | `W` | 1 | 12 |
| 10 | (2,2) | `}` | 12 | 1 |
| 11 | (2,1) | `s` | 6 | 1 |
| 12 | (2,1) | `s` | 6 | 1 |

Ticks 1–5 are the $n^2 + n$ prologue; the westward row reads `<M1W}s`: park $n^2 + n$ in `B`, load 1, swap, halve, send. On tick 12 the man executes `s` — and then steps into the west wall and dies *with the answer still inside the output pipe.* The raw simulator calls this run `error` with **no output at all**. The server disagrees: when the last man stops, in-flight pipe values *drain* — 6 arrives at `O` on tick 13, the case passes, and the clock reads 13. Our `server_compat` judge encodes exactly this (final wall step accepted, output drained); the plain `python -m littleman` judge does not, and scores this machine 0/6.

The sibling `triangle_03.man` is the mirror image: nearly the same program (`M` and `<` swap places, buying the man one extra blank cell to die on), it passes the *plain* judge 6/6 — but the server rejects its layout outright, because it fuses the I and O rooms into `|I|O|`, sharing three wall cells between rooms, which the server forbids. Two near-twin machines, each green under one judge and dead under the other, in opposite directions.

Three lessons: *the footprint is squared, so trade ticks for cells; a man you no longer need costs nothing to kill, because pipes drain after the last man stops; and always gate a wall-trick (or any layout trick) with the server-semantics judge — local green is not server green in either direction.*

# 4 Memory

**The problem.** *Simulate a 100-cell memory. The input is a stream of operations: a READ is the pair* `0 addr`*; a WRITE is the triple* `1 addr value`*. Every cell starts at 0. For each READ, output the current value at* `addr`*; a WRITE produces no output.* Example from the problem statement: the stream `0 5  1 5 10  1 6 9  0 6  0 5` outputs `0 9 10`. Values fit in $[-10^6, 10^6]$; scoring is footprint × ticks.

**Our machine** here is `submissions/memory/memory_04.man` ($37 \times 37$, 24/24 on the server), generated by `src/littleman/memory_packed.py`. It is presented because every mechanism below is documented and asserted by that generator's tests. The team's actual record holder, `memory_10.man` ($30 \times 30$, live score ≈17.2M), is *this same machine* after two generations of room packing — section 4.7 closes with that story.

## 4.1 The two ideas

**Idea 1: memory as a circulating ring.** A littleman machine has no random access — but a pipe is a FIFO, and two pipes plus two rooms make a *ring* on which the whole memory circulates. To touch address $k$ positions ahead of the ring's head you relay $k$ items from the incoming ring pipe to the outgoing one; each relay costs ∼9 ticks. The naive version circulates all 100 cells; profiling it showed ring relays were 93% of all ticks. That measurement is what motivated idea 2.

**Idea 2: pack three cells per word.** A value in $[-10^6, 10^6]$ fits a signed 21-bit field (from $-1\,048\,576$ up to $1\,048\,575$). Three fields live in bits $[0, 21)$, $[21, 42)$, $[42, 63)$ of one signed-64 word:

$$word = (f_0 \wedge M) \;\vee\; ((f_1 \wedge M) \ll 21) \;\vee\; ((f_2 \wedge M) \ll 42),$$

with $M = 2^{21} - 1$. The ring shrinks from 100 items to $\lceil 100/3 \rceil = 34$, and the relay count on the profiled case drops 3.06×. Bit 63 is always zero, so no arithmetic ever wraps, and a freshly zeroed memory is simply the word 0 — the ring seeds with 34 plain zeros instead of long literals.

Address $a$ lives in word $\lfloor a/3 \rfloor$, field $a \bmod 3$ — both produced by *one* `/` operation (which leaves the quotient in `A` and the remainder in `B`). With $shift = 21 \cdot (a \bmod 3)$:

$$\textbf{read:}\quad v \;=\; \big(word \ll (43 - shift)\big) \gg 43$$
$$\textbf{write:}\quad word' \;=\; \big(word \wedge \neg(M \ll shift)\big) \;\vee\; \big((v \wedge M) \ll shift\big)$$

The read needs no mask and no bias: the left shift parks the field's sign bit in bit 63, and the *arithmetic* right shift sign-extends it on the way down. Worked numbers, computed with the real formulas:

| | | |
|---|---|---|
| write 10 at addr 5 | word 1, field 2, shift 42 | word becomes $43\,980\,465\,111\,040 = 10 \ll 42$ |
| read addr 5 | $(word \ll 1) \gg 43$ | $= 10$ |
| write −7 at addr 3 | word 1, field 0, shift 0 | field bits $(-7) \wedge M = 2\,097\,145$ |
| read addr 3 | $(word \ll 43) \gg 43$ | $= -7$ (sign restored by the shift) |
| read addr 5 again | | still 10 — the write of −7 touched only field 0 |

## 4.2 Architecture: five rooms in a pipeline

```
I -> P1 -> HEAD -> P2 -> STATION -> O
                         STATION <-> RELAY     (the 34-word ring)
```

| room | grid position | man | job |
|---|---|---|---|
| P1 | rows 0–4 | (1, 8) | parse one op; split addr into word+shift |
| HEAD | rows 7–10 | (8, 8) | turn word index into ring distance $k$; advance head |
| P2 | rows 13–19 | (14, 8) | precompute the constants the station needs |
| STATION | rows 22–36 | (23, 7) | owns the ring; does the actual read/write |
| RELAY | rows 30–33 (right) | (31, 32) | closes the ring (6-tick lap) |

Each stage passes a small integer stream to the next; all five men run concurrently, so the pipeline overlaps parsing of the next operation with ring work for the current one.

### 4.3 Glossary — the additional operations Memory uses

| | | | |
|---|---|---|---|
| `0`–`9` | `A` ← digit | `‘123‘` | `A` ← literal at closing backtick |
| `W` | swap `A` ↔ `B` | `N` | `A` ← −`A` |
| `-` | `A` ← `A` − `B` | `/` | `A` ← ⌊`A`/`B`⌋, remainder→`B` |
| `%` | `A` ← `A` mod `B` | `&` | `A` ← `A` ∧ `B` (bitwise AND) |
| `\|` | `A` ← `A` ∨ `B` (OR) | `~` | `A` ← `A` ⊕ `B` (XOR) |
| `{` | `A` ← `A` ≪ `B` | `}` | `A` ← `A` ≫ `B` (arithmetic) |
| `b` | `BP` ← `A` | `m` | `BP` ← `BP` − 1 |
| `d` | turn clockwise if `BP` > 0, else straight | `X` | turn by sign of `A` (CW/straight/CCW) |
| `^v<>` | set heading N/S/W/E | `rsM*+H` | as in Triangle |

Two facts do most of the planning work here: `B` is written *only* by `M`, `W`, `/` — so a value parked in `B` survives any amount of `r`/`s` relaying — and `BP` is a countdown register whose `d`-branch turns straight-line corridors into loops.

### 4.4 Stage by stage, with block-graphs

#### P1 — parse and pre-split.

```
(mark L1_8_E) @ r s b r M 3 W / s s W M ‘21‘ * s M (if-bp L2_28_S L1_29_E)
(mark L1_29_E) 0 s W s v v (goto L3_33_S)
(mark L2_28_S) > r s W s v (goto L3_33_S)
(mark L3_33_S) < ^ ^ > (goto L1_8_E)
```

Reads the tag (0=READ, 1=WRITE), forwards it, and *stashes it in the backpack* (`b`) so one `(if-bp ...)` at the end branches the whole room. Reads the address, and `M 3 W /` computes $a/3$ in one stroke: quotient $w$ in `A`, remainder (the field index) in `B`. It emits $w$ twice (HEAD needs it once for the distance, once to advance the head), then $\mathit{shift} = 21 \cdot \mathit{field}$ via `W M ‘21‘ *`. The WRITE arm forwards the value $x$; the READ arm substitutes 0 — so downstream always sees the same shape: [*tag*, $w$, $w$, *shift*, $x$, *shift*].

#### HEAD — ring distance, and a register trick.

```
(mark L8_8_E) @ M r s r - M ‘34‘ W % s r M 1 + M ‘34‘ W % v < M r s r s r s W ^ > (goto L8_8_E)
```

The ring rotates as it is used, so the station never needs an absolute position — only the *distance* $k = (w - hw) \bmod 34$ from the current head word $hw$. HEAD computes that with `-M‘34‘W%`, then the new head $hw' = (w + 1) \bmod 34$. The trick: $hw$ lives in `A` *across laps* — no storage pipe at all. When the tail [*shift*, $x$, *shift*] must be relayed (three `r`/`s` pairs, which clobber `A`), the man first parks $hw'$ in `B` with `M`, relays, and takes it back with `W` — legal precisely because relaying never writes `B`.

#### P2 — precompute the station's constants.

```
(mark L14_8_E) @ r s b r s (if-bp L15_14_S L14_15_E)
(mark L14_15_E) r M ‘43‘ - s r r v < ^ ^ (goto L16_7_N)
(mark L15_14_S) v > r M ‘2097151‘ { M 1 v < N ~ s ‘2097151‘ M r & M r W { s ^ (goto L16_7_N)
(mark L16_7_N) ^ ^ > (goto L14_8_E)
```

The station should do as little arithmetic as possible while it holds the ring, so P2 does it in advance. For a READ it sends [0, $k$, 43−*shift*] (and discards the unused $x$). For a WRITE it builds both write-formula constants: `‘2097151‘ {` makes $M \ll \mathit{shift}$, and the pair `N ~` turns it into $\neg(M \ll \mathit{shift})$ — load −1 (all ones), XOR. Then $(x \wedge M) \ll \mathit{shift}$ via `& {`. It sends [1, $k$, $\neg(M \ll s)$, $(x \wedge M) \ll s$].

**STATION — seed, relay, read or splice.**

```
(mark L23_7_E) @ ‘33‘ b 0 > (goto L23_16_E)
(mark L23_16_E) (s p23_30) v (if-bp L24_16_W L25_17_S)
(mark L24_16_W) m ^ > (goto L23_16_E)
(mark L25_10_N) 3 (goto __wall)
(mark L25_17_S) < v > (goto L26_9_E)
(mark L26_9_E) (r p21_8) (if L25_10_N L26_11_E L27_10_S)
(mark L26_11_E) (r p21_8) b (r p21_8) M v > (goto L27_21_E)
(mark L27_10_S) v v > (r p21_8) (if L29_12_N L30_13_E L31_12_S)
(mark L27_21_E) (r p24_30) (s p23_30) v (if-bp L28_22_W L29_23_S)
(mark L28_7_N) ^ ^ > > (goto L26_9_E)
(mark L28_22_W) m ^ > (goto L27_21_E)
(mark L29_12_N) b b (goto __wall)
(mark L29_23_S) < { M ‘43‘ W } (s p35_5) ^ (goto L28_7_N)
(mark L30_13_E) (r p21_8) M v < < (goto L33_22_W)
(mark L31_12_S) > b m (r p21_8) M > (goto L31_21_E)
(mark L31_21_E) (r p24_30) (s p23_30) v (if-bp L32_22_W L33_23_S)
(mark L32_22_W) m ^ > (goto L31_21_E)
(mark L33_22_W) (r p24_30) & M (r p21_8) | v > (s p23_30) v < ^ ^ ^ ^ ^ ^ ^ (goto L28_7_N)
(mark L33_23_S) < (goto L33_22_W)
(mark __wall) (wall)
```

Reading this graph top to bottom:

- **Seed** (`L23_7_E`): `‘33‘ b 0` then a send–decrement loop — 34 zeros into the ring, i.e. the whole zeroed memory in three cells of program text.
- **Dispatch** (`L26_9_E`): read the tag, `(if NEG ZERO POS)` — 0 goes to the READ arm, 1 to the WRITE arm. (The NEG arm runs into `(wall)`: dead code, since tags are never negative.)
- **READ arm**: `b` loads $k$ into the backpack; `M` parks the decode constant $43 - \mathit{shift}$ in `B`; then the relay loop `(r ring-in)(s ring-out)(if-bp m,loop | exit)` runs $k{+}1$ times. The subtlety: the *target word itself* is also received *and re-sent* — the ring stays intact — and because `r` leaves the received word in `A`, the man exits the loop still holding it. Decode is then `{ M ‘43‘ W }`: shift left by $43 - \mathit{shift}$ (parked in `B` through the whole relay loop), shift right by 43, and the sign-extended field goes to `O`.
- **WRITE arm**: `b m` sets the countdown to $k - 1$, the mask complement is parked in `B`, and the loop relays $k$ words unchanged. The $(k{+}1)$-th word — the target — is received but *not* re-sent: `&` clears the field, `|` splices in the prebuilt $(x \wedge M) \ll \mathit{shift}$, and only the patched word is sent back to the ring. No output is produced, matching the spec.
- Both arms return to the dispatch block; the station is ready for the next operation while upstream rooms are already parsing it.

**RELAY — closing the ring.**

```
(mark L31_32_E) @ > (goto L31_34_E)
(mark L31_34_E) r v < s ^ > (goto L31_34_E)
```

A pipe may not start and end at the same room, so the ring needs a second room: a minimal `r v < s ^ >` lap, 6 ticks per word. Its man runs concurrently with the station's, so the ring keeps flowing while the station computes.

### 4.5 Step by step: the specification's own example

Feeding the example stream `0 5  1 5 10  1 6 9  0 6  0 5` to the real artifact produces exactly the specified output `0 9 10`, at ticks 264, 1011 and 1305. Walking the first two operations through the pipeline, with the packing state:

1. `0 5` (READ 5): P1 splits $5 \to w{=}1$, field 2, *shift*=42; HEAD (head $hw{=}0$) makes $k = (1-0) \bmod 34 = 1$ and advances the head to 2; P2 sends [0, 1, 43−42=1]. The station relays $k{+}1 = 2$ words — word 0 (now the ring's tail) and word 1, keeping word 1 in `A`: it is 0, so $(0 \ll 1) \gg 43 = 0$ is sent to `O`. Output **0** at tick 264.
2. `1 5 10` (WRITE 10 at 5): same split ($w{=}1$, shift 42), but the head is now 2, so $k = (1-2) \bmod 34 = 33$ — the ring must rotate almost a full turn, which is why this operation is the expensive one (∼747 ticks: 33 relays at ∼9 ticks each, plus the pipeline). P2 sends $\neg(M \ll 42)$ and $10 \ll 42 = 43\,980\,465\,111\,040$; the station relays 33 words, patches word 1 to exactly that value, and sends it back. No output.
3. The remaining three operations land the outputs **9** (read of cell 6, field 0 of word 2) and **10** (read of cell 5 again) — the packed word round-trips both fields independently, which is the whole point of the mask-and-splice write.

The cost model this exposes — *an operation costs its ring distance* — is also the machine's weakness: an adversarial address pattern (alternating between two cells 17 words apart) forces half-turns forever. Packing bought a flat 3× on exactly that cost, which is why it was worth the extra arithmetic in P2.

### 4.6 The full program

```
col  0123456789012345678901234567890123456
  0       +---------------------------+
  1  +-+  |>@rsbrM3W/ssWM‘21‘*sMdOsWsv|
  2  |I|>-|^                    >rsWsv|
  3  +-+  |^                        <|
  4       +---------------------------+
  5           v
  6           v
  7       +---------------------------+
  8       |>@Mrsr-M‘34‘W%srM1+M‘34‘W%v|
  9       |^                 WsrsrsrM<|
 10       +---------------------------+
 11           v
 12           v
 13       +------------------------+
 14       |>@rsbrsdrM ‘43‘-srr    v|
 15       |^      v                |
 16       |^      >rM‘2097151‘{M1v |
 17       |^s{WrM&rM‘1517902‘ s~N< |
 18       |^                      <|
 19       +------------------------+
 20         v
 21         v
 22       +----------------------+
 23       |@‘33‘b0 >sv           |>-----v
 24       |        ^md           |<     |
 25       | v        <           ||     |
 26       |>>rXrbrM     v        ||v----<
 27       |^  v         >rsv     |||
 28       |^  v         ^ md     ||>----v
 29       |^s      }W‘34‘M{<     ||  v--<
 30       |^  >rXrM           v  ||+----+
 31       |^    >bmrM   >rsv     |||@>rv|
 32       |^            ^ md     ||| ^s<|
 33       |^ v|r        M&r<  <  ||+----+
 34  +-+  |^ >               sv ||   v
 35  |O|<-<|^                  < |^---<
 36  +-+  +---------------------+
```

Score: bounding box $37 \times 37 \Rightarrow 1369$; server average ∼20 270 ticks over 24/24 cases; score 27 753 851.

### 4.7 The champion is this same machine, packed

The live record holder, `memory_10.man`, has the *identical* architecture — same 7 rooms, 7 pipes, 5 men, and exactly the same operation multiset — with the room interiors repacked (P1 $29 \to 17$ columns wide, HEAD $29 \to 21$, STATION $24 \to 21$) across the hand-tuned `room-packing/` lineage:

| generation | box | footprint | server avg ticks | score |
|---|---|---|---|---|
| `memory_04` | $37 \times 37$ | 1369 | ∼20 270 | 27 753 851 |
| `memory_06` | $34 \times 32$ | 1156 | ∼20 194 | 23 344 360 |
| `memory_10` | $30 \times 30$ | 900 | ∼19 152 | ∼17 236 875 |

Ticks moved 5%; the score moved 1.61×. Every gain in the lineage is *geometry* — the algorithm never changed — which is the squared-footprint lesson from the Triangle section, measured across three generations of one real machine. Everything explained above (the ring, the packing formulas, the five-stage pipeline, the `B`-parking tricks) describes the champion too; it is simply wearing smaller rooms.

# 5 Grade Book: sixteen components with contracts

**The problem.** A roster contains $N = 4..16$ students and $K = 1..4$ subjects. Later rounds contain batches of GET, SET, AVG, and TOP operations. TOP returns the smallest student id among those tied for the highest grade; AVG rounds down. Unlike Memory, this is not naturally one serial ring: each operation names a subject, so our machine gives each subject its own engine and record ring.

**The machine discussed here** is the current live `submissions/gradebook/gradebook_05.man`: $382 \times 307$, footprint $382^2 = 145\,924$, 16 rooms, 30 pipes, and 14 men. Its preserved live result is 20/20 at score 47 115 780 604. The state machines originate in `src/littleman/gradebook.py`; later variants first tightened their layout, then squeezed and staircase-folded the same architecture. `gradebook_05` additionally replaces three fixed worker delays with blocking ring receives and keeps only whole-machine-validated folds. The last fold removed 110 rows and reduced ticks by 21%. It did not change the component protocol described below.

## 5.1 One picture, and the names we will keep

```
input_port --> command_frontend ==broadcast==> subject_engine[1]
                                  |            subject_engine[2]
                                  |            subject_engine[3]
                                  `--------->  subject_engine[4]

subject_engine[j] <--> record_circulator[j]   persistent -N,id,grade ring
subject_engine[j] <--> scratch_circulator[j]  temporary operation state

subject_engine[1] -> [2] -> [3] -> [4] -> command_frontend   acknowledgements
          all four result pipes -> result_arbiter -> output_port
```

These names are semantic rather than geometric, so they survive another repack:

- `input_port` and `output_port`;
- `command_frontend` and `result_arbiter`;
- `subject_engine[1]` through `subject_engine[4]`;
- `scratch_circulator[1..4]` are the rooms historically named `relayB1..4`, in the upper relay tier;
- `record_circulator[1..4]` are the rooms historically named `relayA1..4`, in the lower tier.

All eight circulators are instances of the same physical `RELAY` component. The old A/B labels only described placement; the new names say which stream the room closes.

## 5.2 Why the rooms look empty

The five large rooms were emitted by a correctness-first FSM compiler. A logical block receives its own horizontal band; branches use long vertical tracks; pipe-selection "zones" reserve columns even when most cells in those columns are blank. The rectangles therefore enclose much more air than instructions:

| component | count | envelope | recorded interior ops | wall-inclusive density |
|---|---|---|---|---|
| `command_frontend` | 1 | $385 \times 115$ | 346 | 3.04% |
| `subject_engine[1]` | 1 | $96 \times 154$ | about 686 | about 8.1% |
| `subject_engine[2..4]` | 3 | $94 \times 157$ | about 688–692 | about 8.1% |
| all circulators | 8 | $5 \times 5$ | 8 non-space interior cells | 96% |
| `result_arbiter` | 1 | $384 \times 5$ | 9 | — |
| `input/output_port` | 2 | $3 \times 3$ | one special cell | 100% |

Here "density" includes the enclosing walls. This is the useful visual measure: the front-end and engines really are sparse despite their walls, whereas the $5 \times 5$ circulators are already almost solid. Do not confuse blank cells with the visible `.` cells inside each engine: those dots are executed one-tick delays and are part of the protocol until a test proves otherwise.

### 5.3 The wire protocol

The `command_frontend` turns variable-length input into fixed-width streams. During the roster round it broadcasts

$$[N,\ id_1, g_{1,1}, g_{1,2}, g_{1,3}, g_{1,4},\ \ldots]$$

and fills grades for subjects above $K$ with zero. Each subject engine keeps only its own grade, producing the persistent FIFO

$$[-N,\ id_1, grade_{1,j},\ id_2, grade_{2,j}, \ldots, id_N, grade_{N,j}].$$

The negative count is the end marker and also supplies $N$ for AVG.

Every later operation is broadcast as exactly four values:

$$[op,\ id \text{ or } 0,\ subject,\ value \text{ or } 0].$$

GET and SET carry the real id; AVG and TOP use an id pad. SET carries a real value; the other operations use a value pad. `S` makes the broadcast atomic: it blocks unless all four worker pipes can accept the same token.

Only the engine whose subject number matches executes the operation. The other three drain all four tokens without touching their record rings. After the roster, and after each operation, acknowledgements run

$$engine[1] \longrightarrow engine[2] \longrightarrow engine[3] \longrightarrow engine[4] \longrightarrow frontend.$$

The front-end does not start the next operation before the chain returns. Consequently at most one engine can produce a result for an operation, so the result arbiter's any-ready receive `R` is safe and output order is the input operation order.

### 5.4 Room catalogue: input, control, and output

`input_port`. **Input/output protocol:** accepts the judge's raw round stream and delivers it unchanged over one pipe to `command_frontend`. **State:** none; it is the special $3 \times 3$ `I` room and has no man. **Unit boundary:** exact FIFO preservation across multiple rounds. **Optimization:** the room itself is already minimal; only placement and the two-cell-or-longer connecting pipe can shrink.

`command_frontend`. **Input/output protocol:** consumes raw input plus the final acknowledgement from engine 4; broadcasts the normalized roster and four-token operations over four outgoing pipes. **State:** `B` holds the current roster/operation count, while `BP` selects $K$ during roster loading and selects the operation arm later; no grades persist here. **Unit boundary:** for $K = 1..4$ and boundary $N$, compare its four output streams to a Python normalizer and require it to remain blocked until one ack arrives. **Optimization:** highest priority. It is only 3.04% dense and 385 columns wide. Reorder/merge FSM blocks, or split normalization from a small broadcast fan-out, but preserve all four bindings and atomic `S`.

`result_arbiter`. **Input/output protocol:** receives from any of four engine result pipes with `R` and sends the value to `output_port`. The contract requires zero or one ready input, never two. **State:** only the in-flight value in `A`; no cross-operation state. **Unit boundary:** inject one value through each input in turn, then mixed sequential arrivals, and assert exact ordering and pipe binding. **Optimization:** the $379 \times 5$ room contains only nine interior operations. A compact arbiter or two-level merge tree is promising, although shrinking it alone does not beat the current 382-column bound.

`output_port`. **Input/output protocol:** consumes the arbiter's stream and exposes it to the judge. **State:** none; it is the special $3 \times 3$ `O` room and has no man. **Unit boundary:** GET, AVG, and TOP values must emerge verbatim and SET must emit nothing. **Optimization:** like the input port, optimize only placement and pipe length.

### 5.5 Room catalogue: the four subject engines

All four engines accept the normalized command stream, own one persistent record ring and one temporary scratch ring, optionally produce one result, and send one acknowledgement per operation. Their registers are transient: `BP` selects GET/SET/AVG/TOP; `B` holds such values as the target id, AVG accumulator, or TOP key. Persistent grades live only in the record ring.

`subject_engine[1]`. Consumes subject 1 commands, reads/writes both circulators, and writes the first ack token directly to engine 2; unlike the others it has no predecessor ack input. Unit-test roster load, matching and nonmatching operations, all four opcodes, AVG floor division, and TOP's smallest-id tie. Its $96 \times 154$ envelope is the slightly wider special case. Optimize it as a separate component, preserving its initiator role.

`subject_engine[2]`. The same compute protocol for subject 2, but it must receive engine 1's ack before forwarding an ack to engine 3. Unit-test both the fast nonmatching path and a slow matching operation while the chain waits. Its $94 \times 157$ envelope is representative of the three chained engines.

`subject_engine[3]`. Computes subject 3, receives the ack from engine 2, and forwards it to engine 4. Use the same semantic suite with subject 3 selected, plus a test that other subjects drain without mutating its record FIFO.

`subject_engine[4]`. Computes subject 4 and closes the acknowledgement chain by sending to `command_frontend`. Include $K < 4$ roster padding, $K = 4$ real grades, and delayed final-ack tests; an early ack here would let the parser overlap two operations and corrupt every downstream protocol.

For all four, the optimization target is the room compiler rather than blind whitespace deletion. The existing staircase fold safely merged most carriage-return rows; engine columns remain frozen because they select among command, two ring, ack, and result pipes. A new layout may change those columns only with an operation-cell-to-pipe binding audit. The three explicit 80-dot delays in the source are especially attractive, but removing one is an algorithm change: first replace its settling argument with blocking or a proved ring invariant.

### 5.6 Room catalogue: the eight circulators

Every circulator is the same dense $5 \times 5$ room:

```
+---+
| @v|
|>sv|
|^r<|
+---+
```

Its six-operation lap receives one value and sends it unchanged. It does not interpret the stream; its only register state is the current value in `A`, and the durable FIFO state lives in the two external pipes. Blocking on `r` and `s` supplies backpressure.

| room | exact stream role | unit test and optimization |
|---|---|---|
| `scratch_circulator[1]` | engine 1 scratch-out → unchanged scratch-in; parks ids, SET values, and TOP temporaries while that operation runs | Stream-identity and full/empty blocking tests. Keep the room; shorten its external loop only if engine 1's timing remains exact. |
| `scratch_circulator[2]` | the identical temporary FIFO for engine 2; historical `relayB2` | Run the same parameterized suite with subject 2 and its ack predecessor. The $5 \times 5$ shape is low priority. |
| `scratch_circulator[3]` | the identical temporary FIFO for engine 3; historical `relayB3` | Run the same suite with subject 3. Prefer pipe relocation over changing the already 96%-dense room. |
| `scratch_circulator[4]` | the identical temporary FIFO for engine 4; historical `relayB4` | Test that all scratch traffic finishes before the final ack. Optimize only with the front-end timing gate enabled. |
| `record_circulator[1]` | closes subject 1's persistent $[-N, id, grade, \ldots]$ record FIFO; historical `relayA1` | Rotate boundary $N = 4, 16$ rings, then GET/SET/AVG/TOP. Preserve at least the 33-value maximum parked capacity. |
| `record_circulator[2]` | the persistent subject 2 record FIFO; historical `relayA2` | Same identity/capacity suite, including zero and 100 grades. Shorter pipes are useful only while 33 values still park safely. |
| `record_circulator[3]` | the persistent subject 3 record FIFO; historical `relayA3` | Same suite, plus repeated full rotations to expose an off-by-one marker shift. Room compaction itself has almost no space to win. |
| `record_circulator[4]` | the persistent subject 4 record FIFO; historical `relayA4` | Same suite with $K = 4$ and with padded $K < 4$. Preserve the $-N$ marker and canonical order across every acknowledgement. |

This naming also fixes a common conceptual error: the circulator room is not "the memory." Most values spend their idle time parked at the destination end of its incoming pipe. Unit tests must therefore cover the complete engine–pipe–circulator loop, not execute the five-by-five room in isolation with no capacity model.

### 5.7 Optimization plan enabled by these boundaries

The existing tests prove whole-program behavior, artifact reproduction, public cases, ring capacity, room-byte preservation in the pressed variant, and unchanged pipe roles. They do not yet make each of the sixteen names an independently replaceable module. The next safe sequence is:

1. Extract each room with the same wall-side port offsets and build a harness that supplies and records its named streams. Freeze a protocol oracle before changing geometry.
2. Parameterize the four engines from one semantic test matrix. Test matching/nonmatching subjects, all operations, boundary roster sizes, TOP ties, AVG floor rounding, ring capacity, and ack timing.
3. Optimize `command_frontend` and one `subject_engine` at a time. Compare output streams,

canonical ring state, instruction-cell pipe resolution, and ticks against the old component; then instantiate the three siblings.

4. Recompose all sixteen components. Re-run the public and deterministic oracle workloads, server-compatible layout checks, exact pipe-role maps, and a fresh score benchmark before creating a new version.

The score consequence matters. The current machine is width-bound at 382: the front-end room is 381 cells wide, while the external pipe clearance reaches the final column. More row folding is "banked" but does not reduce the squared footprint. A real next generation must shrink engine *columns*, change their arrangement after shrinking, or both. A previously measured 2×2 arrangement was worse with the present 93-column engine rooms; it should be re-evaluated only after a component compiler makes those widths materially smaller.

# 6 Where everything lives, and how this document was made

| | |
|---|---|
| `submissions/triangle/triangle_02.man` | the live Triangle artifact (rank 1/239) |
| `submissions/memory/memory_04.man` | the packed Memory artifact explained here |
| `submissions/gradebook/gradebook_05.man` | the live Grade Book artifact catalogued here |
| `src/littleman/memory_packed.py` | Memory generator; the packing formulas with tests |
| `src/littleman/gradebook.py` | Grade Book FSMs and original component layout |
| `src/littleman/gradebook_components.py` | Grade Book room contracts and live optimized generator |
| `src/littleman/gradebook_press.py` | named 16-room topology and pipe-role-preserving press |
| `src/littleman/sim.py` | the reference simulator (the semantics authority) |
| `src/littleman/fastsim.py` | the fast executor, proven bit-identical to `sim.py` |
| `src/littleman/decompile.py` | `.man` → block-graph; round-trips the whole corpus |
| `src/littleman/blockgraph.py` | the block-graph parser/interpreter (27 tests) |
| `docs/language-reference.md` | the full operation table, verbatim from the contest |
| `docs/architecture/claude_effects.json` | the *empirical* op-effect table (who writes B, etc.) |

Triangle and Memory grids, block-graphs, traces, and packing numbers were generated from those artifacts by the real toolchain. The traces use `sim.py`, the block-graphs use `decompile.py`, and the packing arithmetic executes the formulas. Grade Book measurements were taken from the checked-in `gradebook_05.man`. Live scores come only from preserved submission records.

To re-verify the headline facts yourself:

```
uv run python -m littleman submissions/triangle/triangle_02.man triangle
uv run python -m littleman submissions/memory/memory_04.man memory
uv run python -m littleman submissions/gradebook/gradebook_05.man gradebook
uv run pytest tests/test_memory_packed.py tests/test_gradebook.py \
  tests/test_gradebook_components.py -q
```